Birla Vishvakarma Mahavidyalaya

Information Technology Department
(An Autonomous CVM Institute)
Vallabh Vidyanagar

Subject: 4IT31 (Project-I), AY: 2026-27

Project Evaluation Report

Group	09
Project Title	Chess Playing Robotic Arm
Faculty Guide	Dr. Keyur Brahmabhatt
Report	SRS
Date of Evaluation	

Group Members

Student ID	**Name**
23IT403	Aashka Shah
23IT431	Veera Agrawal

Comment:

Faculty Guide Signature

1. Introduction

1.1. Purpose of the System

The purpose of the Chess Playing Robotic Arm System is to develop an intelligent robotic system capable of playing chess against a human player by physically moving chess pieces on a standard chessboard.

The system addresses the following needs:

- Integration of robotics, computer vision, and artificial intelligence into a single application.
- Providing an interactive platform for learning chess and robotics.
- Automating chess gameplay with accurate piece recognition and movement.

The system will improve upon existing digital chess systems by offering:

- Physical interaction with real chess pieces.
- Autonomous decision-making using a chess engine.
- Real-time board detection and move execution.
- User-friendly interface for game monitoring.

1.2. Scope of the System

The Chess Playing Robotic Arm System consists of a 6-DOF robotic arm, camera module, Raspberry Pi, Arduino Uno, and a chess engine. The system detects the current board state, calculates the best move, and physically moves the chess pieces.

Key features include:

- Automatic chessboard recognition.
- Human move detection.
- AI-based move generation.
- Physical movement of chess pieces using a robotic arm.
- User interface for game control and monitoring.

2. General Description of the system

2.1. Overall Description

The system is an embedded robotic application that combines computer vision, artificial intelligence, and robotic manipulation. A camera captures the chessboard image and processes it to identify piece positions. The chess engine computes the next move, and the robotic arm executes the move by controlling multiple servo motors.

The system ensures accuracy, reliability, and smooth interaction between hardware and software components.

Additionally, the system has following constraints:

- System should operate on specified voltage mentioned in hardware datasheet.
- The robotic arm should move pieces without disturbing neighboring pieces.
- The camera should provide a clear top view of the chessboard.
- The system should support standard chess rules.
- Real-time response should be maintained during gameplay.

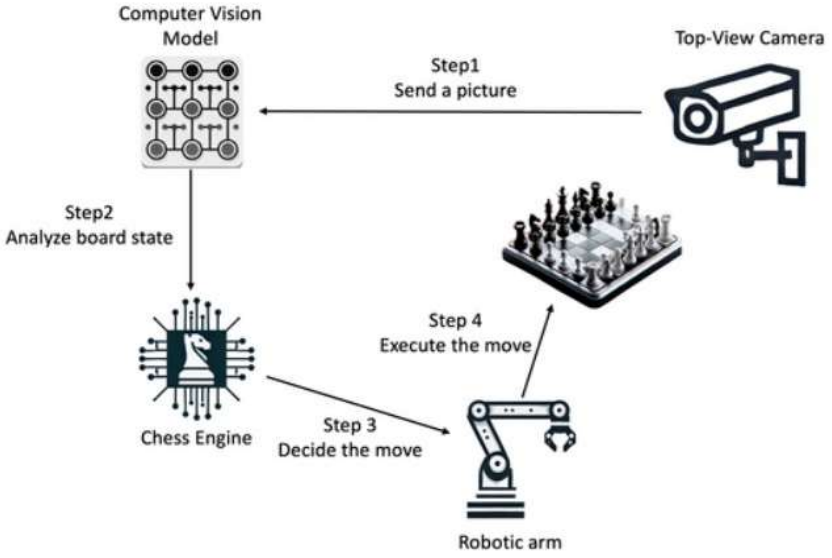

Figure 2.1 Block Diagram for Chess Playing Robotic Arm

2.2. Feasibility Study

2.2.1. Technical Feasibility

- Required technologies such as Raspberry Pi, Arduino, OpenCV, and Stockfish Chess Engine are readily available.
- Servo motors and camera modules can accurately perform the required operations.
- Existing computer vision libraries simplify implementation.
- The system can be upgraded with stronger AI algorithms in the future.

2.2.2. Operational Feasibility

- The system provides an engaging platform for chess players and robotics enthusiasts.
- Human interaction with the system is simple and intuitive.
- The project can be used for educational demonstrations and research purposes.

2.2.3. Economic Feasibility

- The system uses affordable components such as Raspberry Pi, Arduino Uno, and servo motors.
- Open-source software significantly reduces development cost.
- Maintenance cost is low.

3. Functional Requirements

3.1. Module Description

3.1.1. Camera-Based Board Monitoring

The Camera-Based Board Monitoring module is responsible for observing the physical chessboard throughout the game. A top-mounted camera captures high-resolution images of the board after each player's move. Using image processing techniques with OpenCV, the module detects the chessboard boundaries, divides the board into 64 squares, and identifies the position of every chess piece. The captured board state is converted into a digital representation that is used by the remaining modules. Proper camera calibration and consistent lighting ensure accurate detection while minimizing recognition errors caused by shadows or reflections.

3.1.2. Move Detection Module

The Move Detection module identifies the move made by the human player by comparing the current board state with the previous one. It analyzes the changes in the positions of chess pieces to determine the source and destination squares of the move. The detected move is then validated according to standard chess rules before being accepted. This module also recognizes special moves such as castling, pawn promotion, en passant captures, and piece captures. If an invalid move is detected, the system notifies the player and waits until a legal move is performed before proceeding.

3.1.3. Chess AI Engine

The Chess AI Engine acts as the decision-making component of the system. After receiving the updated board state, it communicates with the Stockfish chess engine to calculate the optimal move. The board position is converted into Forsyth-Edwards Notation (FEN), which is provided as input to the engine. Based on the selected difficulty level, the engine evaluates numerous possible moves and returns the best move within the allotted computation time. The module also handles game logic such as check, checkmate, stalemate, draw conditions, and legal move generation, ensuring that the game strictly follows official chess rules.

3.1.4. Coordinate Mapping Module

The Coordinate Mapping module runs on the Raspberry Pi and serves as the interface between the chess engine and the robotic arm. It converts the chess engine's output (source and destination squares) into physical coordinates that correspond to the chessboard. Each square (A1 to H8) is mapped to predefined Cartesian coordinates based on the board dimensions and the robotic arm's workspace. The module determines the pick-up position, destination position, and intermediate safe positions required for smooth movement. It also assigns predefined storage locations for captured pieces. Using these coordinates, the Raspberry Pi computes the required joint angles through inverse kinematics and sends the calculated servo angle commands to the Arduino Uno for execution. This module ensures accurate positioning and collision-free movement of the robotic arm.

3.1.5. Robotic Arm Control Module

The Robotic Arm Control module is implemented using the Arduino Uno, which is responsible for controlling the six servo motors of the robotic arm. It receives the servo angle commands generated by the Raspberry Pi and produces the corresponding PWM control signals to drive each servo motor. Based on these commands, the robotic arm moves to the source square, grips the chess piece using the end-effector, lifts it to a safe height, transports it to the destination square, and releases it accurately. For capture moves, the Arduino first executes the sequence to remove the opponent's piece to the designated capture area before placing the moving piece on the target square. By accurately following the commands received from the Raspberry Pi, the Arduino ensures smooth, precise, and reliable movement of the robotic arm throughout the game.

3.1.6. Game Management Module

The Game Management module coordinates the overall operation of the chess-playing robotic system. It maintains the current game state, records the sequence of moves, tracks player turns, and determines the progress of the match. The module initializes a new game with the standard chess setup, updates the board after every move, and stores game history for future reference or analysis. It also handles exceptional situations such as illegal moves, communication failures, or interrupted gameplay by providing appropriate notifications and recovery options. This module acts as the central controller that synchronizes communication among the camera, move detection, chess engine, coordinate mapping, and robotic arm control modules to ensure smooth and uninterrupted gameplay.

3.2. Functions of various user of the system

There are mainly two users of the system:

Admin:

- Manage users
- Monitor game listings and activities
- Configure hardware settings
- Calibrate camera and robotic arm
- Update software and chess engine

Client:

- Start and stop games.
- Make chess moves.
- Select difficulty level.
- Monitor game progress.
- Perform game analysis.

4. Non-Functional Requirements

4.1. Security

- Only authorized users can modify system settings.
- Game data and configurations are protected.

4.2. Reliability

- The system should operate continuously without crashes.
- Accurate piece movement should exceed 95%.

4.3. Availability

- System should be available whenever powered on.
- Real-time responses should occur within a few seconds.

4.4. Maintainability

- Modular software architecture.
- Easy replacement of hardware components.

4.5. Portability

- Software should run on Raspberry Pi OS.
- Code should be portable to other Linux-based systems.

4.6. Reusability

- Vision module can be reused in other board-game projects.
- Robotic arm control module can be reused in pick-and-place applications.

5. Interface Requirements

5.1. GUI

5.1.1. Home Screen

- Start Game button
- Difficulty Selection
- Game History
- System status display

5.1.2. Game Screen

- Clock Timer
- Error notifications

5.2. Hardware Interface

- Raspberry Pi 4
- Arduino Uno
- 6-DOF Robotic Arm
- Servo Motors and Driver
- External Power Supply

5.3. Software Interface

- Programming Language: Python and C
- Operating System: Raspberry Pi OS
- Computer Vision Library: OpenCV
- Chess Engine: Stockfish
- Communication Protocol: Serial UART
- Arduino IDE for motor control programming

6. Data Dictionary

6.1. Table Name: user_table

Sr. No.	Name	Datatype	Constraint	Description
1	user_id	INT	Primary key auto_increment	Unique user ID
2	password	PASSWORD(10)	Not null	Encrypted Password
3	email	VARCHAR(10)	Not null	User email id
4	username	VARCHAR(10)	Not null	User name as player

Figure 6.1 user_table

6.2. Table Name: game_table

Sr. No.	Name	Datatype	Constraint	Description
1	game_id	INT	Primary key auto_increment	Unique game ID
2	user_id	INT	Foreign key	Reference to user table
3	start_time	DATETIME	Not Null	Game start time
4	end_time	DATETIME	Not Null	Game end time
5	difficulty	VARCHAR	Not null	Easy/Medium/Hard
6	result	VARCHAR(20)	Not null	Win/Loss/Draw

Figure 6.2 game_table

6.3. Table Name: move_table

Sr. No.	Name	Datatype	Constraint	Description
1	move_id	INT	Primary key auto_increment	Unique move ID
2	game_id	INT	Foreign key	Reference to game table
3	move_number	INT	Not Null	Move sequence number
4	move	VARCHAR(20)	Not Null	Move displayed in standard chess notation
5	player	VARCHAR(20)	Not Null	Human or Robot

Table 6.3 move_table

6.4. Table Name: motor_log

Sr. No.	Name	Datatype	Constraint	Description
1	log_id	INT	Primary key auto_increment	Unique log ID
2	motor_id	INT	Not null	Servo motor ID
3	angle	FLOAT	Not null	Motor Angle
4	timestamp	DATETIME	Not Null	Execution Time
5	status	VARCHAR(20)	Not null	Success/Failure

Table 6.4 motor_log